THE COMPLETE DSA & CS FUNDAMENTALS

INTERVIEW PREPARATION GUIDE

NeetCode 150 + All Core CS Subjects

Crack Any Company: FAANG, Startups & Beyond

HOW TO USE THIS GUIDE

This guide is divided into two major parts:

- Part 1 – DSA Topics & NeetCode 150 Questions: Learn every topic needed to solve coding interview questions at top companies.
- Part 2 – Core CS Subjects: OOP, DBMS, OS, Networking, System Design, Computer Architecture, and more.

TIP: Follow the topics in the order listed. Each topic builds on the previous one. Do not skip to hard problems without mastering easy ones first.

NOTE: NeetCode 150 is a curated list of 150 LeetCode problems that cover all patterns tested at FAANG and top product companies.

Study Roadmap

Phase	Duration	Focus
Phase 1	Weeks 1-3	Arrays, Strings, Hashing, Two Pointers, Sliding Window
Phase 2	Weeks 4-6	Stack, Binary Search, Linked List, Trees
Phase 3	Weeks 7-9	Tries, Heap, Backtracking, Graphs
Phase 4	Weeks 10-12	Dynamic Programming, Greedy, Bit Manipulation, Math
Phase 5	Weeks 13-16	System Design, OOP, DBMS, Networking, OS review

PART 1: DATA STRUCTURES & ALGORITHMS

NeetCode 150 — Complete Topic Breakdown

TOPIC 1: Arrays & Hashing

What is it?

Arrays are contiguous memory blocks that store elements of the same type. Hashing uses a hash function to map keys to indices in a hash table for O(1) average lookups.

Why is it important?

- Every interview starts with an array or string problem. Mastering this is non-negotiable.
- Hash Maps (dictionaries) reduce time complexity from O(n^2) to O(n) in many problems.
- Used as a building block for almost all other DSA topics.

Key Concepts to Learn

- Array traversal, indexing, slicing
- HashMap / HashSet — storing counts, checking existence
- Frequency map pattern — count occurrences of characters/numbers
- Two-pass technique — first pass store info, second pass use it
- Prefix sum — cumulative sum array for range query O(1)

> **TIP:** When you see 'find two numbers that sum to target', immediately think HashMap. Store complement = target - num.

NeetCode 150 Questions — Arrays & Hashing

#	Problem	Difficulty	Key Insight
1	Contains Duplicate	Easy	Use HashSet; if element already in set, return true
2	Valid Anagram	Easy	Count character frequencies using HashMap or array[26]
3	Two Sum	Easy	HashMap: store {value: index}; check complement at each step
4	Group Anagrams	Medium	Sort each word as key; group by sorted key in HashMap
5	Top K Frequent Elements	Medium	HashMap for freq + bucket sort or heap
6	Encode and Decode Strings	Medium	Use length prefix format: '5#hello'

#	Problem	Difficulty	Key Insight
7	Product of Array Except Self	Medium	Two passes: prefix product, then suffix product; no division
8	Valid Sudoku	Medium	3 HashSets per row, col, 3x3 box; use (row//3, col//3) as box key
9	Longest Consecutive Sequence	Medium	HashSet; only start counting from sequence start (num-1 not in set)

TOPIC 2: Two Pointers

What is it?

The two pointers technique uses two indices (pointers) that move through an array or string simultaneously. They can move from both ends toward the center, or move at different speeds.

When to Use Two Pointers

- Array is sorted (or can be sorted)
- You need to find a pair or triplet satisfying some condition
- You need to compare characters from both ends (palindrome check)
- Removing duplicates in-place from sorted array

Patterns

Pattern 1: Opposite Ends

Left pointer starts at 0, right pointer at end. Move them inward based on conditions.

Pattern 2: Same Direction (Fast & Slow)

Both start at 0 but one moves faster. Used for finding duplicates in linked lists, middle of list.

> **TIP:** For 3Sum: Fix one element, then use two pointers on the rest. Sort first!

NeetCode 150 Questions — Two Pointers

#	Problem	Difficulty	Key Insight
1	Valid Palindrome	Easy	Two pointers from ends; skip non-alphanumeric; compare lowercase
2	Two Sum II (Sorted)	Medium	Left+right pointers; if sum too big move right left, too small move left right
3	3Sum	Medium	Sort + fix i, use two pointers for rest; skip duplicates carefully
4	Container With Most Water	Medium	Move the pointer with shorter height; greedy proof: can only improve
5	Trapping Rain Water	Hard	Max from left and right; water at i = min(maxL, maxR) - height[i]

TOPIC 3: Sliding Window

What is it?

A sliding window is a subarray or substring of variable or fixed size that moves through the data. Instead of re-computing for every window from scratch, we add the new element and remove the old one.

Types

Fixed Size Window

Window size k is constant. Slide by removing leftmost and adding rightmost.

Variable Size Window

Expand right pointer until condition violated, then shrink from left. Used for 'longest/shortest subarray with condition'.

> **TIP:** The template: expand right, when condition breaks, shrink left. Track max/min window size throughout.

NeetCode 150 Questions — Sliding Window

#	Problem	Difficulty	Key Insight
1	Best Time to Buy & Sell Stock	Easy	Track min price so far; max profit = current - minPrice
2	Longest Substring Without Repeating	Medium	HashSet window; shrink left when duplicate found
3	Longest Repeating Char Replacement	Medium	Window valid if (window size - max freq char) <= k
4	Permutation in String	Medium	Fixed window size len(s1); compare char frequency maps
5	Minimum Window Substring	Hard	HashMap; track 'have' vs 'need' counts; shrink when valid
6	Sliding Window Maximum	Hard	Monotonic deque; keep only indices of decreasing elements

TOPIC 4: Stack

What is it?

A stack is a LIFO (Last In, First Out) data structure. Elements are pushed onto the top and popped from the top. In most languages, a list/array is used to implement a stack.

When to Use a Stack

- Matching parentheses / brackets (balanced check)
- Next Greater Element problems
- Monotonic stack for finding nearest smaller/larger elements
- Evaluating expressions (postfix, prefix)
- Undo operations, DFS iterative implementation

Monotonic Stack Pattern

A monotonic stack maintains elements in increasing or decreasing order. When a new element violates the monotonic property, pop elements and process them. Very powerful for 'next greater/smaller element' problems.

NeetCode 150 Questions — Stack

#	Problem	Difficulty	Key Insight
1	Valid Parentheses	Easy	Push open brackets; on close, check if top of stack matches
2	Min Stack	Medium	Two stacks: one normal, one tracking current minimum
3	Evaluate Reverse Polish Notation	Medium	Push numbers; on operator, pop two, compute, push result
4	Generate Parentheses	Medium	Backtracking; track open & close counts; open<n, close<open
5	Daily Temperatures	Medium	Monotonic decreasing stack; store indices; pop when warmer found
6	Car Fleet	Medium	Sort by position; compute time to reach target; stack of fleet times
7	Largest Rectangle in Histogram	Hard	Monotonic increasing stack; pop when shorter bar found; calc area

TOPIC 5: Binary Search

What is it?

Binary search works on sorted arrays by repeatedly halving the search space. At each step, compare the middle element with the target. Time complexity: O(log n).

The Template (Most Important)

Use this template to avoid off-by-one errors:

```
lo = 0, hi = len(arr) - 1
while lo <= hi:
    mid = (lo + hi) // 2
    if arr[mid] == target: return mid
    elif arr[mid] < target: lo = mid + 1
    else: hi = mid - 1
```

Advanced: Binary Search on Answer

Instead of searching in an array, binary search on the answer space. E.g., 'Find minimum speed such that X is possible.' Check if a value is feasible, binary search for the optimal.

NeetCode 150 Questions — Binary Search

#	Problem	Difficulty	Key Insight
1	Binary Search	Easy	Standard template; return -1 if lo > hi
2	Search a 2D Matrix	Medium	Treat matrix as 1D array; row = mid//cols, col = mid%cols
3	Koko Eating Bananas	Medium	Binary search on speed 1..max; check feasibility
4	Find Minimum in Rotated Sorted Array	Medium	If mid > right, min is in right half; else left half
5	Search in Rotated Sorted Array	Medium	Determine which half is sorted; check if target in that half
6	Time Based Key-Value Store	Medium	Store list of (timestamp, value); binary search for largest t <= target
7	Median of Two Sorted Arrays	Hard	Binary search on partition point; ensure left max <= right min

TOPIC 6: Linked List

What is it?

A linked list is a chain of nodes where each node holds a value and a pointer to the next node. Unlike arrays, linked lists allow O(1) insertion/deletion at known positions but O(n) access by index.

Must-Know Techniques

- Dummy node — simplifies edge cases (empty list, head deletion)
- Fast & Slow pointers — detect cycle, find middle
- Reversing a linked list — iterative (3 pointers: prev, curr, next)
- Merge two sorted lists — pointer manipulation
- Two pointers with gap — find Nth from end

NeetCode 150 Questions — Linked List

#	Problem	Difficulty	Key Insight
1	Reverse Linked List	Easy	prev=None, curr=head; iterate: next=curr.next, curr.next=prev, prev=curr, curr=next
2	Merge Two Sorted Lists	Easy	Dummy node; compare and attach smaller node each step
3	Reorder List	Medium	Find mid (slow/fast), reverse second half, merge alternately
4	Remove Nth Node From End	Medium	Two pointers N apart; when right hits None, remove left.next
5	Copy List with Random Pointer	Medium	HashMap {original: copy}; two passes — create nodes, then set pointers
6	Add Two Numbers	Medium	Traverse both, add digits + carry; handle unequal lengths
7	Linked List Cycle	Easy	Floyd's: fast moves 2, slow moves 1; cycle if they meet
8	Find Duplicate Number	Medium	Treat as linked list (value = next index); Floyd's cycle detection
9	LRU Cache	Medium	HashMap + Doubly Linked List; O(1) get and put
10	Merge K Sorted Lists	Hard	Min-heap of (val, listIndex); always extract min and advance
11	Reverse Nodes in K-Group	Hard	Reverse k nodes at a time; recursively handle rest

TOPIC 7: Trees

What is it?

A tree is a hierarchical data structure with nodes connected by edges. The topmost node is the root. Binary trees have at most 2 children per node. Binary Search Trees (BST) maintain the property: left < node < right.

Traversal Types

Traversal	Order	Use Case
Inorder (L-Root-R)	Left, Root, Right	BST inorder = sorted order
Preorder (Root-L-R)	Root, Left, Right	Copy tree, serialization
Postorder (L-R-Root)	Left, Right, Root	Delete tree, evaluate expression
Level Order (BFS)	Level by level	Shortest path, level-wise operations

Key Recursive Patterns

- Return value from left and right subtrees, combine at root
- Pass values down to children (top-down: max depth, path sum)
- Return values up from children (bottom-up: diameter, balanced check)

NeetCode 150 Questions — Trees

#	Problem	Difficulty	Key Insight
1	Invert Binary Tree	Easy	Recursively swap left and right children
2	Maximum Depth of Binary Tree	Easy	1 + max(depth(left), depth(right))
3	Diameter of Binary Tree	Easy	At each node: diameter = leftHeight + rightHeight; track global max
4	Balanced Binary Tree	Easy	Height function returns -1 if unbalanced; propagate up
5	Same Tree	Easy	Recursively compare values and structure
6	Subtree of Another Tree	Easy	isSameTree at each node using DFS
7	Lowest Common Ancestor of BST	Medium	If both > node: go right; both < node: go left; else: node is LCA
8	Binary Tree Level Order Traversal	Medium	BFS with queue; track level size to separate levels
9	Binary Tree Right Side View	Medium	BFS; take last node of each level

#	Problem	Difficulty	Key Insight
10	Count Good Nodes in Binary Tree	Medium	DFS passing current max; node is good if val >= max
11	Validate Binary Search Tree	Medium	Pass min/max bounds; each node must be in (min, max)
12	Kth Smallest Element in BST	Medium	Inorder traversal; count k elements; return kth
13	Construct Tree from Preorder+Inorder	Medium	First of preorder is root; find in inorder to split left/right
14	Binary Tree Maximum Path Sum	Hard	At each node: gain = val + max(0,left) + max(0,right); return val + max(0, one side)
15	Serialize and Deserialize Binary Tree	Hard	Preorder with None markers; use queue for deserialization

TOPIC 8: Tries (Prefix Trees)

What is it?

A Trie (pronounced 'try') is a tree-like data structure used to store strings where each node represents a character. It's extremely efficient for prefix-based searching, autocomplete, and spell checkers.

Structure

- Each node has: children (dict or array of 26), isEnd (bool)
- Insert: traverse/create nodes for each character
- Search: traverse; return true if all chars found and isEnd=True
- StartsWith: traverse; return true if all chars found (ignore isEnd)

NeetCode 150 Questions — Tries

#	Problem	Difficulty	Key Insight
1	Implement Trie	Medium	TrieNode with children dict and isEnd flag; insert/search/prefix methods
2	Design Add and Search Words Data Structure	Medium	Trie + wildcard '.' means try all children recursively
3	Word Search II	Hard	Build Trie from words; DFS on board tracking current Trie node

TOPIC 9: Heap / Priority Queue

What is it?

A heap is a complete binary tree satisfying the heap property. Min-heap: parent <= children (root is smallest). Max-heap: parent >= children (root is largest). Operations: O(log n) insert, O(log n) delete, O(1) peek.

Key Uses

- Top K elements — maintain a heap of size K
- K-way merge — merge K sorted arrays/lists
- Dijkstra's shortest path
- Scheduling / task management

TIP: Python's heapq is a min-heap. For max-heap, negate values: push -val, pop and negate result.

NeetCode 150 Questions — Heap

#	Problem	Difficulty	Key Insight
1	Kth Largest Element in Stream	Easy	Min-heap of size k; root is kth largest
2	Last Stone Weight	Easy	Max-heap; smash two heaviest; push remainder if nonzero
3	K Closest Points to Origin	Medium	Min-heap by distance; or max-heap of size k
4	Kth Largest Element in Array	Medium	Min-heap size k; or QuickSelect O(n) average
5	Task Scheduler	Medium	Greedy: always schedule most frequent; idle if needed
6	Design Twitter	Medium	Per-user tweet list; merge K sorted lists with heap for feed
7	Find Median from Data Stream	Hard	Two heaps: maxHeap (lower half) + minHeap (upper half); balance sizes

TOPIC 10: Backtracking

What is it?

Backtracking is a systematic way to iterate through all possible configurations of a search space. At each step, make a choice, recurse, then undo the choice (backtrack) if it doesn't lead to a solution.

The Template

```
def backtrack(start, current_path):
    if base_case: add to result; return
    for choice in choices:
        make choice (add to path)
        backtrack(next_state, current_path)
        undo choice (remove from path)
```

Subsets vs Permutations vs Combinations

Type	Order Matters?	Reuse?	Typical Start
Subsets	No	No	Start from 'i+1' each recursion
Permutations	Yes	No	Use visited[] array; try all at each level
Combinations	No	Sometimes	Start from 'i' (reuse) or 'i+1' (no reuse)

NeetCode 150 Questions — Backtracking

#	Problem	Difficulty	Key Insight
1	Subsets	Medium	At each index: include or exclude element; 2^n subsets total
2	Combination Sum	Medium	Reuse allowed; start from same index; stop when sum > target
3	Permutations	Medium	Use visited array; pick any unvisited element each level
4	Subsets II (with duplicates)	Medium	Sort first; skip duplicate elements at same recursion level
5	Combination Sum II	Medium	Sort; one use per element; skip duplicates at same level
6	Word Search	Medium	DFS from each cell; mark visited; restore on backtrack
7	Palindrome Partitioning	Medium	At each index, try all palindrome substrings; recurse on rest

#	Problem	Difficulty	Key Insight
8	Letter Combinations of Phone Number	Medium	Map digits to letters; build combinations recursively
9	N-Queens	Hard	Track cols, diagonals, anti-diagonals; place queens row by row

TOPIC 11: Graphs

What is it?

A graph is a collection of nodes (vertices) connected by edges. Graphs can be directed or undirected, weighted or unweighted. They model networks, relationships, maps, dependencies.

Representations

- Adjacency List: dict of {node: [neighbors]}. Space efficient for sparse graphs.
- Adjacency Matrix: 2D array. Space O(V^2). Good for dense graphs.
- Edge List: list of (u, v) pairs. Used in algorithms like Kruskal's MST.

Core Algorithms

Algorithm	Type	Time Complexity	Use Case
DFS	Traversal	O(V + E)	Connected components, cycle detection, topological sort
BFS	Traversal	O(V + E)	Shortest path (unweighted), level-order
Dijkstra's	Shortest Path	O((V+E) log V)	Shortest path in weighted graph (non-negative weights)
Union-Find	Connectivity	O(alpha(n)) ~ O(1)	Detect cycle, MST (Kruskal's), connected components
Topological Sort	Ordering	O(V + E)	Task scheduling, course prerequisites
Bellman-Ford	Shortest Path	O(VE)	Handles negative weights, detect negative cycles

NeetCode 150 Questions — Graphs

#	Problem	Difficulty	Key Insight
1	Number of Islands	Medium	DFS/BFS from each '1'; mark visited; count starts
2	Clone Graph	Medium	HashMap {original: clone}; DFS/BFS to build clone
3	Max Area of Island	Medium	DFS returning size; track max
4	Pacific Atlantic Water Flow	Medium	Reverse BFS from each ocean; find intersection of reachable cells
5	Surrounded Regions	Medium	Mark border-connected O's safe; flip rest to X
6	Rotting Oranges	Medium	Multi-source BFS from all rotten oranges; track time
7	Walls and Gates	Medium	Multi-source BFS from all gates (0s); fill distances

#	Problem	Difficulty	Key Insight
8	Course Schedule	Medium	Detect cycle in directed graph using DFS with 3-color (unvisited/visiting/visited)
9	Course Schedule II	Medium	Topological sort (Kahn's algo or DFS post-order)
10	Graph Valid Tree	Medium	Connected + no cycle: DFS/BFS, or edges = n-1 and connected
11	Number of Connected Components	Medium	DFS/BFS count, or Union-Find
12	Redundant Connection	Medium	Union-Find; first edge that creates a cycle is redundant
13	Word Ladder	Hard	BFS; each word is a node; connect words differing by 1 char

Advanced Graph — Dijkstra's Template

```
heap = [(0, start)]  # (cost, node)
dist = {start: 0}
while heap:
    cost, node = heappop(heap)
    for neighbor, weight in graph[node]:
        new_cost = cost + weight
        if new_cost < dist.get(neighbor, inf):
            dist[neighbor] = new_cost
            heappush(heap, (new_cost, neighbor))
```

TOPIC 12: Dynamic Programming (DP)

What is it?

Dynamic Programming solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation. DP is either top-down (memoization) or bottom-up (tabulation).

How to Identify DP

- 'Find maximum/minimum...'
- 'Count the number of ways...'
- 'Is it possible...'
- Problem has optimal substructure and overlapping subproblems

The Framework

- Step 1: Define the state — what does dp[i] or dp[i][j] represent?
- Step 2: Write the recurrence relation
- Step 3: Identify base cases
- Step 4: Determine order of computation
- Step 5: Optimize space if needed (rolling array, etc.)

DP Categories

Category	Classic Problems	State Pattern
1D DP	Climbing Stairs, House Robber	dp[i] = f(dp[i-1], dp[i-2])
2D DP	Unique Paths, Edit Distance	dp[i][j] = f(dp[i-1][j], dp[i][j-1])
Knapsack	0/1 Knapsack, Coin Change	dp[i][w] = include or exclude item i
LCS/LIS	Longest Common Subsequence	Compare two sequences character by character
Interval DP	Burst Balloons, Matrix Chain	dp[l][r] = optimal for subarray l..r
Tree DP	Max Path Sum, House Robber III	State at each node based on children

NeetCode 150 Questions — 1D DP

#	Problem	Difficulty	Recurrence
1	Climbing Stairs	Easy	dp[i] = dp[i-1] + dp[i-2] (Fibonacci)
2	Min Cost Climbing Stairs	Easy	dp[i] = cost[i] + min(dp[i-1], dp[i-2])

#	Problem	Difficulty	Recurrence
3	House Robber	Medium	dp[i] = max(dp[i-1], dp[i-2] + nums[i])
4	House Robber II (Circular)	Medium	Run House Robber on [0..n-2] and [1..n-1]; take max
5	Longest Palindromic Substring	Medium	Expand around center; or 2D DP dp[i][j]=palindrome?
6	Palindromic Substrings	Medium	Count all palindromes by expanding around each center
7	Decode Ways	Medium	dp[i] = ways to decode s[0..i]; check 1-digit and 2-digit
8	Coin Change	Medium	dp[amount] = 1 + min(dp[amount-coin]) for each coin
9	Maximum Product Subarray	Medium	Track both max and min (negatives flip sign)
10	Word Break	Medium	dp[i] = True if s[0..i] can be segmented
11	Longest Increasing Subsequence	Medium	dp[i] = 1 + max(dp[j]) where j < i and nums[j] < nums[i]
12	Partition Equal Subset Sum	Medium	0/1 Knapsack on target = total_sum // 2

NeetCode 150 Questions — 2D DP

#	Problem	Difficulty	Recurrence
1	Unique Paths	Medium	dp[i][j] = dp[i-1][j] + dp[i][j-1]
2	Longest Common Subsequence	Medium	If match: dp[i][j]=dp[i-1][j-1]+1; else max(left,up)
3	Best Time to Buy/Sell Stock with Cooldown	Medium	States: holding, sold, idle; transitions at each day
4	Coin Change II	Medium	dp[i][a] = ways using first i coins to make amount a
5	Target Sum	Medium	dp[i][s] = ways to assign +/- to first i elements for sum s
6	Interleaving String	Medium	dp[i][j] = s3[0..i+j] formed by s1[0..i] and s2[0..j]
7	Longest Increasing Path in Matrix	Hard	DFS with memoization; cannot revisit cells
8	Distinct Subsequences	Hard	dp[i][j] = count of s subsequences equaling t
9	Edit Distance	Hard	dp[i][j] = min ops to convert s[0..i] to t[0..j]
10	Burst Balloons	Hard	dp[l][r] = max coins from bursting all in range l..r
11	Regular Expression Matching	Hard	dp[i][j] = s[0..i] matches p[0..j]?

TOPIC 13: Greedy Algorithms

What is it?

Greedy algorithms make the locally optimal choice at each step, hoping to find a global optimum. Unlike DP, no backtracking or exploring other choices. Always prove greedy works by exchange argument or staying ahead argument.

NeetCode 150 Questions — Greedy

#	Problem	Difficulty	Greedy Strategy
1	Maximum Subarray (Kadane's)	Medium	If running sum becomes negative, reset to 0
2	Jump Game	Medium	Track maxReach; if i > maxReach, cannot proceed
3	Jump Game II	Medium	Track farthest reachable; increment jumps when reach current boundary
4	Gas Station	Medium	If total gas < total cost: impossible; else start from first station where runningGas < 0 + 1
5	Hand of Straights	Medium	Sort + greedy consume: always start from smallest remaining
6	Merge Triplets to Form Target	Medium	Include triplet if no component exceeds target; union components
7	Partition Labels	Medium	Track last occurrence; extend partition until all chars in window settled
8	Valid Parenthesis String	Medium	Track min and max possible open count; '(' inc, ')' dec, '*' both

TOPIC 14: Intervals

What is it?

Interval problems deal with ranges [start, end]. Common operations: merge overlapping intervals, insert a new interval, find non-overlapping intervals, check if person can attend all meetings.

Key Insight

Two intervals [a,b] and [c,d] overlap if and only if: a <= d AND c <= b. They merge into [min(a,c), max(b,d)].

NeetCode 150 Questions — Intervals

#	Problem	Difficulty	Key Insight
1	Insert Interval	Medium	Add all before, merge overlapping, add all after
2	Merge Intervals	Medium	Sort by start; merge if curr.start <= prev.end
3	Non-Overlapping Intervals	Medium	Greedy: sort by end; greedily keep interval ending earliest
4	Meeting Rooms	Easy	Sort by start; if any overlap → false
5	Meeting Rooms II	Medium	Min-heap of end times; if top <= start, reuse room; else add room
6	Minimum Interval to Include Each Query	Hard	Sort intervals + queries; sweep with min-heap

TOPIC 15: Bit Manipulation

What is it?

Bit manipulation directly operates on the binary representation of numbers using bitwise operators: AND (&), OR (|), XOR (^), NOT (~), left shift (<<), right shift (>>).

Essential Tricks

Operation	Code	Explanation
Check if bit i is set	n & (1 << i)	Result nonzero if bit i is 1
Set bit i	n | (1 << i)	Turns bit i to 1
Clear bit i	n & ~(1 << i)	Turns bit i to 0
Toggle bit i	n ^ (1 << i)	Flips bit i
Check if power of 2	n & (n-1) == 0	Powers of 2 have exactly 1 set bit
Remove lowest set bit	n & (n-1)	Famous Brian Kernighan trick
Get lowest set bit	n & (-n)	Isolates rightmost 1 bit
XOR same values cancel	a ^ a = 0	Find unique number in array

NeetCode 150 Questions — Bit Manipulation

#	Problem	Difficulty	Key Insight
1	Single Number	Easy	XOR all numbers; pairs cancel; unique remains
2	Number of 1 Bits	Easy	n & (n-1) removes lowest set bit; count iterations
3	Counting Bits	Easy	dp[i] = dp[i >> 1] + (i & 1)
4	Reverse Bits	Easy	Extract LSB, shift into result, shift n right; 32 times
5	Missing Number	Easy	XOR all indices and values; missing cancels out
6	Sum of Two Integers	Medium	a + b = (a^b) + ((a&b)<<1); iterate until carry=0
7	Reverse Integer	Medium	Extract digits; check overflow before multiplying

TOPIC 16: Math & Geometry

NeetCode 150 Questions — Math & Geometry

#	Problem	Difficulty	Key Insight
1	Rotate Image	Medium	Transpose matrix, then reverse each row
2	Spiral Matrix	Medium	4 pointers: top, bottom, left, right; shrink after each pass
3	Set Matrix Zeroes	Medium	Use first row/col as markers; scan markers to zero out
4	Happy Number	Easy	Cycle detection: slow/fast on digit-square-sum function
5	Plus One	Easy	Carry propagation; handle all-9 edge case
6	Pow(x, n)	Medium	Fast exponentiation: x^n = (x^(n//2))^2; handle negative n
7	Multiply Strings	Medium	digit[i]*digit[j] contributes to result[i+j] and result[i+j+1]
8	Detect Squares	Medium	HashMap of points; for each pair, check 2 completing squares

QUICK REFERENCE: Time & Space Complexity

Data Structure	Access	Search	Insert	Delete	Space
Array	O(1)	O(n)	O(n)	O(n)	O(n)
Dynamic Array	O(1)	O(n)	O(1) amort	O(n)	O(n)
Linked List	O(n)	O(n)	O(1)	O(1)	O(n)
Stack / Queue	O(n)	O(n)	O(1)	O(1)	O(n)
Hash Table	N/A	O(1) avg	O(1) avg	O(1) avg	O(n)
BST (balanced)	O(log n)	O(log n)	O(log n)	O(log n)	O(n)
Heap	O(1) top	O(n)	O(log n)	O(log n)	O(n)
Trie	O(m)	O(m)	O(m)	O(m)	O(n*m)
Graph (adj list)	O(1)	O(V+E)	O(1)	O(E)	O(V+E)

Algorithm	Best	Average	Worst	Space
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)
Binary Search	O(1)	O(log n)	O(log n)	O(1)
BFS / DFS	O(1)	O(V+E)	O(V+E)	O(V)

PART 2: CORE COMPUTER SCIENCE SUBJECTS

Essential Knowledge for Every Developer Interview

CS SUBJECT 1: Object-Oriented Programming (OOP)

What is OOP?

OOP is a programming paradigm that organizes software design around objects (data + behavior) rather than functions and logic. The four core pillars are: Encapsulation, Abstraction, Inheritance, and Polymorphism.

The 4 Pillars — DEEP EXPLANATION

1. Encapsulation

Encapsulation bundles data (attributes) and methods that operate on that data within a single unit (class). It restricts direct access to some components using access modifiers (private, protected, public), exposing only what is necessary.

- Why: Prevents accidental data corruption; hides implementation details
- How: Getters/setters, private fields in Java/C++, _ prefix in Python
- Example: A BankAccount class keeps balance private; only deposit() and withdraw() methods can change it

2. Abstraction

Abstraction hides complex implementation details and shows only the essential features. It allows you to think at a higher level without worrying about implementation specifics.

- Why: Reduces complexity; allows focus on what an object does, not how
- How: Abstract classes, interfaces, abstract methods
- Example: A Car class has a start() method — you don't need to know about fuel injection or combustion

3. Inheritance

Inheritance allows a class (child/subclass) to inherit properties and behaviors from another class (parent/superclass). Promotes code reuse and creates an is-a relationship.

- Why: Code reuse; hierarchical classification; extendability
- Types: Single, Multiple (C++, Python), Multilevel, Hierarchical, Hybrid
- Example: Dog extends Animal — Dog inherits eat() and sleep() but adds bark()
- Method Resolution Order (MRO) in Python: C3 linearization

4. Polymorphism

Polymorphism means 'many forms.' The same interface can work with different types. Two types: compile-time (method overloading) and runtime (method overriding).

- Compile-time (Static): Method Overloading — same name, different parameters
- Runtime (Dynamic): Method Overriding — child class redefines parent method
- Why: Flexibility and substitutability; code works with future types
- Example: shape.draw() works for Circle, Square, Triangle — each overrides draw()

Other Key OOP Concepts

Class vs Object

Class = Blueprint (template). Object = Instance of a class. Creating an object allocates memory and calls the constructor (__init__ in Python, constructor method in Java).

Abstract Class vs Interface

Feature	Abstract Class	Interface
Can have fields	Yes	Only constants (Java)
Can have constructor	Yes	No
Multiple inheritance	No (Java)	Yes
Methods with body	Yes	Default methods (Java 8+)
Use when	Partial implementation shared	Define a contract/behavior

SOLID Principles

- S — Single Responsibility: A class should have only one reason to change
- O — Open/Closed: Open for extension, closed for modification
- L — Liskov Substitution: Subclasses should be substitutable for their base class
- I — Interface Segregation: Clients should not be forced to depend on interfaces they don't use
- D — Dependency Inversion: Depend on abstractions, not concrete implementations

Design Patterns

Category	Pattern	Problem Solved
Creational	Singleton	Ensure only one instance of a class
Creational	Factory Method	Create objects without specifying exact class
Creational	Builder	Construct complex objects step by step
Structural	Adapter	Allow incompatible interfaces to work together
Structural	Decorator	Add responsibilities to objects dynamically
Structural	Facade	Provide simplified interface to a complex subsystem
Behavioral	Observer	Notify multiple objects when state changes (pub/sub)
Behavioral	Strategy	Define family of algorithms, make them interchangeable
Behavioral	Iterator	Sequential access to collection elements

IMPORTANT: Interview Hot Topics: Polymorphism vs Overloading, Abstract class vs Interface, Singleton pattern, SOLID principles, Composition vs Inheritance.

CS SUBJECT 2: Database Management Systems (DBMS)

What is DBMS?

A DBMS is software that manages the creation, maintenance, and use of databases. It provides an interface between users/applications and the actual data stored. Examples: MySQL, PostgreSQL, Oracle, MongoDB.

Database Architecture

- Physical Level: How data is physically stored on disk (files, blocks, pages)
- Logical Level: What data is stored — tables, relationships (schema)
- View Level: User-specific views — what each user/app sees

SQL — Structured Query Language

DDL — Data Definition Language (structure)

CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE, RENAME

DML — Data Manipulation Language (data)

SELECT, INSERT, UPDATE, DELETE

DCL — Data Control Language (permissions)

GRANT, REVOKE

TCL — Transaction Control Language

COMMIT, ROLLBACK, SAVEPOINT

Joins — Most Asked in Interviews

Join Type	Returns	Example Use Case
INNER JOIN	Rows matching in BOTH tables	Orders with matching customers
LEFT JOIN	All from left + matching from right (NULL if no match)	All customers, even without orders
RIGHT JOIN	All from right + matching from left (NULL if no match)	All orders, even orphaned
FULL OUTER JOIN	All from both; NULL where no match	Complete comparison of two tables
CROSS JOIN	Cartesian product (every row x every row)	Generate combinations
SELF JOIN	Table joined with itself	Employee-manager relationships

Normalization

Normalization reduces data redundancy and improves data integrity by organizing data into tables and following normal forms.

Normal Form	Rule	Removes
1NF	Atomic values; no repeating groups; primary key	Multi-valued attributes
2NF	1NF + no partial dependency on composite key	Partial dependencies
3NF	2NF + no transitive dependency	Transitive dependencies
BCNF	Stronger 3NF: every determinant is a candidate key	Anomalies in 3NF tables
4NF	BCNF + no multi-valued dependencies	Multi-valued dependencies

ACID Properties (Transactions)

- Atomicity: Transaction is all-or-nothing. If any step fails, entire transaction is rolled back.
- Consistency: Transaction brings database from one valid state to another. All constraints maintained.
- Isolation: Concurrent transactions behave as if sequential. Isolation levels: Read Uncommitted, Read Committed, Repeatable Read, Serializable.
- Durability: Once committed, data persists even after system failure. Achieved through write-ahead logging (WAL).

Indexing

An index is a data structure (usually B-Tree or Hash) that speeds up data retrieval at the cost of storage space and slower writes.

- Clustered Index: Data physically stored in index order. Only one per table (usually primary key).
- Non-Clustered Index: Separate structure pointing to data rows. Multiple allowed per table.
- Composite Index: Index on multiple columns. Order matters for queries.
- Covering Index: Index includes all columns needed by a query — no table lookup needed.

SQL vs NoSQL

Feature	SQL (Relational)	NoSQL
Schema	Fixed, predefined	Dynamic, flexible
Data model	Tables with rows/columns	Documents, Key-Value, Graph, Column
Scaling	Vertical (scale up)	Horizontal (scale out)
Transactions	Full ACID	BASE (Basically Available, Soft state, Eventually consistent)
Use case	Complex queries, consistency critical	High throughput, flexible schema, massive scale

Feature	SQL (Relational)	NoSQL
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra, DynamoDB

Other Key DBMS Concepts

- Views: Virtual tables based on SELECT queries. Simplify complex queries. Updatable views are limited.
- Stored Procedures: Pre-compiled SQL code stored on server. Faster, reusable, secure.
- Triggers: Auto-executed code in response to INSERT/UPDATE/DELETE on a table.
- Deadlock: Two transactions each waiting for the other to release a lock. Prevention: timeout, deadlock detection, lock ordering.
- Sharding: Horizontal partitioning across multiple database instances. Range-based, hash-based, or directory-based.

CS SUBJECT 3: Operating Systems (OS)

What is an OS?

An Operating System is system software that manages hardware resources and provides services for computer programs. It sits between hardware and application software. Examples: Linux, Windows, macOS.

Process vs Thread

Feature	Process	Thread
Definition	Program in execution; has its own memory space	Lightweight unit of execution within a process
Memory	Own virtual address space	Shares process memory (heap, code, globals)
Communication	IPC (pipes, sockets, shared memory)	Shared memory directly (but needs sync)
Creation cost	High (fork)	Low (thread_create)
Crash impact	Does not affect other processes	Can crash entire process
Context switch	Expensive	Cheaper

Process States

- New: Process being created
- Ready: Waiting to be assigned to CPU
- Running: Instructions being executed
- Waiting/Blocked: Waiting for I/O or event
- Terminated: Execution finished

CPU Scheduling Algorithms

Algorithm	Type	Key Property
FCFS (First Come First Serve)	Non-preemptive	Simple; convoy effect with long processes
SJF (Shortest Job First)	Non-preemptive	Optimal average waiting time; starvation possible
SRTF (Shortest Remaining Time First)	Preemptive	Optimal but requires knowing burst time
Round Robin	Preemptive	Time quantum; fair; widely used in time-sharing
Priority Scheduling	Both	Higher priority runs first; starvation — solved with aging
Multilevel Queue	Both	Different queues for different process categories

Deadlock

A deadlock is a situation where a set of processes are each waiting for resources held by another process in the set — none can proceed.

Coffman's Conditions (all 4 must hold for deadlock):

- 1. Mutual Exclusion — resource cannot be shared
- 2. Hold and Wait — process holds resource and waits for another
- 3. No Preemption — resource cannot be forcibly taken
- 4. Circular Wait — P1 waits for P2, P2 waits for P3, P3 waits for P1

Prevention: Break any one condition. Avoidance: Banker's Algorithm. Detection + Recovery: Kill processes or preempt resources.

Memory Management

- Paging: Divide memory into fixed-size pages; eliminate external fragmentation
- Segmentation: Divide by logical units (code, stack, heap); can have external fragmentation
- Virtual Memory: Process can use more memory than physically available; OS pages in/out
- Page Replacement: FIFO, LRU, Optimal (Belady's); LRU most commonly used
- Thrashing: Excessive page faults causing CPU to spend more time paging than executing

Synchronization

- Race Condition: Multiple threads access shared data concurrently; outcome depends on timing
- Critical Section: Code that accesses shared resources; only one thread should execute at a time
- Mutex: Binary lock; only owning thread can release it
- Semaphore: Counting lock; can allow N threads simultaneously
- Monitor: High-level sync; encapsulates mutex + condition variables
- Deadlock vs Livelock vs Starvation: All are synchronization problems; livelock = active but no progress

CS SUBJECT 4: Computer Networks

The OSI Model — 7 Layers

Layer	Name	Function	Protocols
7	Application	User-facing network services	HTTP, HTTPS, FTP, SMTP, DNS
6	Presentation	Data format, encryption, compression	SSL/TLS, JPEG, MPEG
5	Session	Establish, manage, terminate sessions	NetBIOS, PPTP
4	Transport	End-to-end delivery, flow control, error recovery	TCP, UDP
3	Network	Logical addressing and routing	IP, ICMP, OSPF, BGP
2	Data Link	MAC addressing, framing, error detection	Ethernet, Wi-Fi, ARP
1	Physical	Bits over medium (cables, radio waves)	Ethernet cable, fiber, Wi-Fi

TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Guaranteed delivery with ACKs	No guarantee; best effort
Ordering	In-order delivery	No ordering
Speed	Slower (overhead)	Faster (less overhead)
Error checking	Yes (checksum + retransmit)	Checksum only (no retransmit)
Use case	HTTP, email, file transfer	Video streaming, DNS, gaming, VoIP
Header size	20-60 bytes	8 bytes

HTTP / HTTPS

- HTTP (HyperText Transfer Protocol): Application layer protocol for web communication. Stateless.
- HTTPS: HTTP + TLS/SSL encryption. Prevents man-in-the-middle attacks.
- HTTP Methods: GET (retrieve), POST (create), PUT (replace), PATCH (partial update), DELETE (remove)
- HTTP Status Codes: 1xx informational, 2xx success, 3xx redirect, 4xx client error, 5xx server error
- HTTP/1.1 vs HTTP/2: HTTP/2 adds multiplexing, header compression, server push, binary framing

DNS — Domain Name System

DNS translates human-readable domain names (google.com) to IP addresses (142.250.80.46). It's a distributed, hierarchical system.

- Recursive resolver → Root servers → TLD servers (.com, .org) → Authoritative nameserver
- DNS record types: A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail), NS (nameserver), TXT (verification)

Key Network Concepts

- IP Addressing: IPv4 (32-bit, e.g., 192.168.1.1), IPv6 (128-bit). Subnetting with CIDR notation.
- NAT (Network Address Translation): Maps multiple private IPs to one public IP.
- Firewall: Filters network traffic based on rules (port, IP, protocol).
- Load Balancer: Distributes traffic across multiple servers. Round-robin, least connections, IP hash.
- CDN (Content Delivery Network): Geographically distributed servers cache content closer to users.
- WebSockets: Full-duplex communication over single TCP connection. Used for real-time apps.

CS SUBJECT 5: System Design

The Framework for Any System Design Interview

1. Clarify requirements — functional (what it does) and non-functional (scale, latency, availability)
2. Estimate scale — DAU, QPS, storage needed
3. Define the API — endpoints, request/response format
4. High-level design — draw major components
5. Deep dive — dig into the most interesting/critical components
6. Identify bottlenecks and trade-offs

Core Building Blocks

Horizontal vs Vertical Scaling

- Vertical: Add more CPU/RAM to one machine. Simple but has limits.
- Horizontal: Add more machines. Requires load balancing, stateless design.

Caching

Cache stores frequently accessed data in fast memory to reduce latency and database load.

- Cache-aside (Lazy loading): App checks cache first; on miss, loads from DB and populates cache
- Write-through: Write to cache and DB simultaneously
- Write-back: Write to cache; async write to DB (faster but risk of loss)
- Eviction policies: LRU (most common), LFU, FIFO, TTL-based
- Tools: Redis (in-memory key-value, pub/sub, sorted sets), Memcached (simpler, multi-threaded)

Databases

- Relational (SQL): Strong consistency, ACID, complex queries. Use for financial data, user records.
- NoSQL Document (MongoDB): Flexible schema, JSON-like. Use for catalogs, user profiles.
- Key-Value (Redis, DynamoDB): O(1) lookup. Use for sessions, caching, counters.
- Column-Family (Cassandra): High write throughput, wide rows. Use for time-series, IoT, logs.
- Graph (Neo4j): Nodes and edges. Use for social networks, recommendations.
- Search (Elasticsearch): Full-text search with inverted index. Use for product search, logs.

Message Queues

Decouple producer and consumer. Enable async processing, buffering, load leveling.

- Kafka: High-throughput distributed log. Partitioned, replicated. Used for streaming.
- RabbitMQ: Traditional message broker. Supports AMQP. For task queues.
- SQS (AWS): Managed queue. Standard vs FIFO.

Consistent Hashing

Used in distributed systems to assign data to nodes with minimal redistribution when nodes are added/removed. Nodes and data are mapped to a ring; each key goes to the first node clockwise.

CAP Theorem

In a distributed system, you can only guarantee 2 of 3 properties simultaneously:

- C — Consistency: All nodes see the same data at the same time
- A — Availability: Every request gets a response (not necessarily the latest)
- P — Partition Tolerance: System continues despite network partitions

> **NOTE:** Network partitions are inevitable in distributed systems. So real choice is CP vs AP. Cassandra = AP, HBase = CP, most SQL = CA (no partition tolerance assumed).

Common System Design Questions

System	Key Components	Main Challenges
URL Shortener	Hash function, DB (key-value), redirect	Hash collision, custom aliases, analytics
Rate Limiter	Token bucket/sliding window, Redis counter	Distributed rate limiting, race conditions
News Feed	Fan-out on write vs read, timeline DB	Celebrity problem, ordering, caching
Chat System	WebSockets, message queue, storage	Online status, message ordering, delivery receipts
Design Twitter	Tweet service, fan-out, search, CDN	Hotspot users, real-time updates, scale
Design YouTube	Video upload/transcoding, CDN, recommendations	Storage, streaming, encoding, thumbnails
Design Uber	Geospatial index, matching, maps	Location updates, driver matching, surge pricing
Distributed Cache	Consistent hashing, replication, eviction	Cache coherence, thundering herd, warm-up

CS SUBJECT 6: Computer Architecture

Von Neumann Architecture

The foundational model of modern computers: CPU, Memory (RAM), Input/Output, and a bus connecting them. The CPU fetches instructions from memory and executes them sequentially (fetch-decode-execute cycle).

CPU Components

- ALU (Arithmetic Logic Unit): Performs arithmetic and logical operations
- CU (Control Unit): Directs flow of data between CPU, memory, and I/O
- Registers: Ultra-fast storage inside CPU (PC, IR, MAR, MDR, accumulator)
- Cache: L1 (fastest, smallest), L2, L3 (slowest, largest). Reduces memory access time.
- Pipeline: Overlap multiple instructions (fetch, decode, execute, write-back simultaneously)

Memory Hierarchy

Level	Type	Speed	Size	Cost
1	CPU Registers	< 1 ns	Bytes	Very high
2	L1 Cache	~1 ns	32-64 KB	High
3	L2 Cache	~5 ns	256 KB - 1 MB	High
4	L3 Cache	~20 ns	4-32 MB	Medium-high
5	RAM (Main Memory)	~100 ns	4-64 GB	Medium
6	SSD	~100 us	256 GB - 2 TB	Low
7	HDD	~10 ms	1-20 TB	Very low

Key Concepts

- RISC vs CISC: RISC (ARM) has simple instructions, executes fast. CISC (x86) has complex instructions, more work per instruction.
- Little Endian vs Big Endian: Byte order in memory. Intel CPUs are little-endian (LSB first).
- Pipelining Hazards: Data hazard (dependency), Control hazard (branches), Structural hazard (resource conflict)
- Cache Coherence: In multi-core, ensures all caches have consistent view of memory (MESI protocol)
- Branch Prediction: CPU guesses next instruction path; modern CPUs are 95%+ accurate

CS SUBJECT 7: Compilers & Programming Languages

Compilation Phases

- Lexical Analysis: Source code → tokens (keywords, identifiers, operators)
- Syntax Analysis (Parsing): Tokens → Parse tree/AST (Abstract Syntax Tree) using grammar
- Semantic Analysis: Type checking, scope checking, semantic correctness
- Intermediate Code Generation: AST → 3-address code or IR
- Optimization: Constant folding, dead code elimination, loop optimization
- Code Generation: IR → machine code

Interpreted vs Compiled vs JIT

Type	How	Examples	Trade-off
Compiled	Translate entire program before run	C, C++, Go, Rust	Fast execution; platform-specific
Interpreted	Translate and execute line by line	Python, Ruby, PHP	Portable; slower execution
JIT (Just-In-Time)	Compile hot paths at runtime	Java (JVM), JavaScript V8, C#	Best of both; warm-up time

Memory Layout of a Program

- Text/Code Segment: Compiled instructions (read-only)
- Data Segment: Initialized global and static variables
- BSS Segment: Uninitialized global/static variables (zero-initialized)
- Heap: Dynamic memory allocation (malloc, new); grows upward
- Stack: Function call frames, local variables, return addresses; grows downward

Garbage Collection

- Reference Counting: Object deleted when ref count reaches 0 (Python). Problem: circular references.
- Mark-and-Sweep: Mark all reachable objects; sweep and collect unmarked (Java, Go)
- Generational GC: Most objects die young; collect young gen frequently (Java G1, Python)

CS SUBJECT 8: Software Engineering Principles

Software Development Life Cycle (SDLC)

- Planning: Define scope, feasibility, resources
- Requirements: Gather functional and non-functional requirements
- Design: System architecture, database design, UI/UX
- Implementation: Coding, code reviews, unit tests
- Testing: Integration, system, performance, user acceptance testing (UAT)
- Deployment: CI/CD pipelines, release management
- Maintenance: Bug fixes, feature updates, monitoring

Agile & Scrum

- Agile: Iterative, incremental development; embrace change; working software over documentation
- Scrum: Sprint (1-4 weeks), Daily Standup, Sprint Planning, Sprint Review, Retrospective
- Kanban: Continuous flow; WIP limits; visualize workflow on board
- CI/CD: Continuous Integration (merge often, automated tests) + Continuous Delivery/Deployment

Version Control — Git

- Core commands: init, clone, add, commit, push, pull, fetch, merge, rebase, stash
- Branching strategies: Git Flow, GitHub Flow, Trunk-Based Development
- Merge vs Rebase: Merge preserves history; rebase rewrites history for cleaner log
- Cherry-pick: Apply a specific commit to another branch
- Resolve conflicts: Manual edit → git add → git commit

Testing Types

Type	What is Tested	Scope	Tools
Unit Testing	Individual functions/methods	Smallest unit	Jest, JUnit, pytest
Integration Testing	Interaction between components	Multiple units	Postman, REST Assured
System Testing	Complete application	End-to-end	Selenium, Cypress
Performance Testing	Speed, stability under load	Non-functional	JMeter, k6, Locust
TDD (Test-Driven Dev)	Write tests before code	All levels	Any unit test framework

CS SUBJECT 9: Cloud Computing & DevOps

Cloud Service Models

Model	What You Manage	Examples
IaaS (Infrastructure as a Service)	OS, runtime, apps (provider gives hardware)	AWS EC2, Azure VMs, GCP Compute
PaaS (Platform as a Service)	Only applications (provider manages infra + OS)	Heroku, AWS Elastic Beanstalk, App Engine
SaaS (Software as a Service)	Nothing (just use the software)	Gmail, Salesforce, Dropbox
FaaS/Serverless	Just functions (no server management)	AWS Lambda, Google Cloud Functions

Containers & Orchestration

- Docker: Containerizes apps with all dependencies. Image = template, Container = running instance.
- Dockerfile: Instructions to build an image. FROM, RUN, COPY, CMD, EXPOSE.
- Docker Compose: Define multi-container apps with docker-compose.yml.
- Kubernetes (K8s): Orchestrates containers at scale. Pod (smallest unit), Deployment, Service, Ingress.
- Key K8s concepts: Self-healing, auto-scaling, rolling deployments, load balancing.

CI/CD Pipeline

Code → Build → Test → Staging Deploy → Production Deploy

- Tools: GitHub Actions, Jenkins, CircleCI, GitLab CI, ArgoCD
- Artifacts: Docker images, JAR files, bundles pushed to registry
- Blue-Green Deployment: Two identical environments; switch traffic to new version
- Canary Release: Route small % of traffic to new version; monitor; gradually increase

CS SUBJECT 10: Security & Cryptography

Core Security Concepts

- Authentication vs Authorization: AuthN (who are you?) vs AuthZ (what can you do?)
- Symmetric Encryption: Same key for encrypt/decrypt. Fast. AES-256. For data at rest.
- Asymmetric Encryption: Public key encrypts, private key decrypts. Slower. RSA, ECC. For key exchange.
- Hashing: One-way function. MD5 (broken), SHA-256, bcrypt (for passwords — adds salt + slow).
- TLS/SSL: Asymmetric for key exchange, symmetric for bulk data, MAC for integrity.
- JWT (JSON Web Token): Header.Payload.Signature. Stateless authentication token.

Common Vulnerabilities (OWASP Top 10)

Vulnerability	Description	Prevention
SQL Injection	Malicious SQL in input fields	Parameterized queries, ORM
XSS (Cross-Site Scripting)	Inject scripts into web pages viewed by others	Escape output, Content Security Policy
CSRF	Trick user into submitting unintended request	CSRF tokens, SameSite cookies
Broken Authentication	Weak passwords, session fixation	MFA, secure session management
Sensitive Data Exposure	Unencrypted passwords, PII in logs	Encrypt at rest and transit, mask data
Insecure Deserialization	Exploit serialized object to execute code	Validate input, use safe formats

FINAL: Cracking Any Interview — Strategy Guide

Coding Interview Strategy (for DSA)

- Step 1 — Understand: Read problem twice. Ask clarifying questions. State assumptions.
- Step 2 — Examples: Walk through examples manually. Identify edge cases (empty, single element, all same).
- Step 3 — Brute Force: Always start with naive solution. State its time and space complexity.
- Step 4 — Optimize: Think about patterns — can I use hashing, two pointers, DP? Trade time for space?
- Step 5 — Code: Write clean, readable code. Name variables well. Think aloud.
- Step 6 — Test: Trace through examples. Check edge cases. Dry run on paper.

Complexity Analysis Tips

- Nested loops = usually O(n^2); but if inner loop is shrinking → might be O(n log n)
- Recursive with branching factor b and depth d = O(b^d)
- Sorting = O(n log n) minimum for comparison-based; O(n) with counting/radix sort
- HashMap operations = O(1) average, O(n) worst case (hash collisions)

Behavioral Interview — STAR Method

Structure every behavioral answer with: Situation, Task, Action, Result.

- Situation: Set the context. When/where/who/what.
- Task: What was your responsibility or the challenge?
- Action: What did YOU specifically do? Use 'I' not 'we'.
- Result: Quantify the outcome. What changed? What was the impact?

Most Common Behavioral Questions

- Tell me about yourself — 2-minute pitch: background, key achievements, why this role
- Greatest weakness — pick real weakness with genuine improvement effort
- Conflict with colleague — focus on resolution and learning
- Failure — show ownership, learning, changed behavior
- Why this company — research culture, product, mission; be specific

Last Mile Tips

TIP: Practice coding on paper or whiteboard — not just IDE. Syntax errors slow you down in interviews.

TIP: For system design: always talk through trade-offs. Interviewers want to see how you think, not a perfect answer.

TIP: For behavioral: prepare 5-7 strong stories that can answer most questions from different angles.

IMPORTANT: One month before interview: focus on Mock interviews — time yourself solving problems under pressure. Sites: Pramp, Interviewing.io, LeetCode contests.

Company-Wise Focus

Company	DSA Focus	System Design?	CS Depth?
Google	Hard LeetCode, graphs, DP	Yes (senior roles)	Medium
Amazon	Medium LeetCode, arrays, trees	Yes (LPs matter)	Medium
Microsoft	Medium LeetCode, OOP design	Yes	Medium
Meta (Facebook)	Arrays, graphs, BFS/DFS	Yes	Medium
Apple	Medium LeetCode, OOP	Sometimes	Low-Medium
Startups	Easy-Medium, practical	High-level	Low
Service Companies (TCS, Infosys)	Easy LeetCode, basic DS	Rare	High (CS theory)

Good luck! Consistency > Intensity. Solve 2-3 problems daily and review theory weekly.